

Лабораторный практикум 13. Фролов А.А.

Добавление тестовой процедуры Imit для отладки процедур переписки сектора:

```
5  start:
6      jmp Imit
7
8      ; Начальный адрес области памяти
9      Address dw 100h
10
11     ; Данные программы: 256 байтов
12     ; Первая строка сделана для примера: буквы, управляющие символы и байты ≥ 80h
13  Sector db 'A','B','C','D','E','F','G',00h,20h,1Fh,80h,81h,82h,83h,84h,85h
14          db 16 dup(11h)
15          db 16 dup(12h)
16          db 16 dup(13h)
17          db 16 dup(14h)
18          db 16 dup(15h)
19          db 16 dup(16h)
20          db 16 dup(17h)
21          db 16 dup(18h)
22          db 16 dup(19h)
23          db 16 dup(1Ah)
24          db 16 dup(1Bh)
25          db 16 dup(1Ch)
26          db 16 dup(1Dh)
27          db 16 dup(1Eh)
28          db 16 dup(1Fh)
29
30     ; -----
31     ; Тестовая процедура для лабораторной 11
32     ; -----
33  Imit:
34      call Init_sector
35
36      mov ah,1
37      int 21h
38
39      call N_sector
40
41      mov ah,1
42      int 21h
43
44      call Prev_sector
45
46      mov ah,1
47      int 21h
48
49      call N_sector
50
51      mov ah,1
52      int 21h
53
54      call Next_sector
55
56      mov ah,1
57      int 21h
58
59      int 20h ; выход в DOS
60
```

Добавление процедуры Copy_sector для копирования 256 байт памяти:

```
;-----  
; Копирование сектора 256 байт  
; Вход:  
; SI - начальный адрес сектора-источника  
; DI - начальный адрес сектора-приемника  
;-----  
Copy_sector:  
    push cx  
    pushf  
  
    cld  
    mov cx,256  
    rep movsb  
  
    popf  
    pop cx  
    ret
```

Изменили disp_sector так как программой теперь управляет Imit

```
;-----  
; Вывод сектора: 256 байтов = 16 строк по 16 байтов  
;-----  
disp_sector:  
    call Clear_screen  
  
    xor dx,dx      ; DX = номер первого байта строки  
    mov cx,16      ; всего 16 строк  
  
sector_loop:  
    call disp_line  ; вывести одну строку  
    call Send_crlf  ; переход на новую строку  
  
    add dx,16       ; следующая строка начинается через 16 байтов  
    loop sector_loop  
    | Ctrl+L to chat  
    ret
```

Добавление процедуры Write_sector для записи сектора в память

```
; -----  
; Копирование сектора 256 байт  
; Вход:  
; SI - начальный адрес сектора-источника  
; DI - начальный адрес сектора-приемника  
; -----  
Copy_sector:  
    push cx  
    pushf  
  
    cld  
    mov cx,256  
    rep movsb  
  
    popf  
    pop cx  
    ret  
  
; -----  
; Запись содержимого Sector в сектор памяти  
; Адрес сектора хранится в переменной Address  
; -----  
Write_sector:  
    push si  
    push di  
  
    mov si,offset Sector  
    mov di,[Address]  
    call Copy_sector  
  
    pop di  
    pop si  
    ret
```

Добавление процедуры Init_sector для загрузки начального сектора памяти

```
; -----  
; Запись содержимого Sector в сектор памяти  
; Адрес сектора хранится в переменной Address  
; -----  
Write_sector:  
    push si  
    push di  
  
    mov si,offset Sector  
    mov di,[Address]  
    call Copy_sector  
  
    pop di  
    pop si  
    ret  
  
; -----  
; Инициализация сектора  
; Address = 0000h  
; Sector = содержимое сектора 0  
; -----  
Init_sector:  
    push si  
    push di  
  
    mov word ptr [Address],0  
  
    mov si,0  
    mov di,offset Sector  
    call Copy_sector  
  
    call disp_sector  
  
    pop di  
    pop si  
    ret
```

Добавление процедуры Prev_sector для перехода к предыдущему сектору памяти

```
; -----  
; Загрузка предыдущего сектора памяти  
; Если текущий сектор 00, остаёмся на нём  
; -----  
Prev_sector:  
    push si  
    push di  
  
    mov ax,[Address]  
  
    cmp ax,0  
    je Prev_no_change  
  
    sub ax,100h  
    mov [Address],ax  
  
Prev_no_change:  
    mov si,[Address]  
    mov di,offset Sector  
    call Copy_sector  
  
    call disp_sector  
  
    pop di  
    pop si  
    ret
```

Добавление процедуры Next_sector для перехода к следующему сектору памяти

```
; -----  
; Загрузка следующего сектора памяти  
; Если текущий сектор FF, остаёмся на нём  
; -----  
Next_sector:  
    push si  
    push di  
  
    mov ax,[Address]  
  
    cmp ax,0FF00h  
    je Next_no_change  
  
    add ax,100h  
    mov [Address],ax  
  
Next_no_change:  
    mov si,[Address]  
    mov di,offset Sector  
    call Copy_sector  
  
    call disp_sector  
  
    pop di  
    pop si  
    ret
```

Создание файла Kbd_io.asm с процедурами ввода шестнадцатеричного числа

```
1  ; -----  
2  ; Ввод одной шестнадцатеричной цифры  
3  ; Выход:  
4  ; AL = значение цифры от 0 до 15  
5  ; -----  
6  Read_digit_hex:  
7      mov ah,01h  
8      int 21h  
9  
10     cmp al,'0'  
11     jnb Read_digit_hex  
12  
13     cmp al,'9'  
14     jbe KH_09  
15  
16     cmp al,'A'  
17     jnb KH_SMALL_CHECK  
18  
19     cmp al,'F'  
20     jbe KH_AF_BIG
```

```

21
22     KH_SMALL_CHECK:
23         cmp al,'a'
24         jnb Read_digit_hex
25
26         cmp al,'f'
27         jbe KH_AF_SMALL
28
29         jmp Read_digit_hex
30
31     KH_09:
32         sub al,'0'
33         ret
34
35     KH_AF_BIG:
36         sub al,'A'
37         add al,10
38         ret
39
40     KH_AF_SMALL:
41         sub al,'a'
42         add al,10
43         ret
44
45
46     ; -----
47     ; Ввод двухзначного шестнадцатеричного числа
48     ; Выход:
49     ; AL = введённый байт
50     ; Например: ввод 05 → AL = 05h
51     ;           ввод FF → AL = FFh
52     ; -----
53     Read_byte_hex:
54         push bx
55
56         call Read_digit_hex
57         mov bl,al
58         shl bl,4
59
60         call Read_digit_hex
61         add bl,al
62
63         mov al,bl
64
65         pop bx
66         ret

```

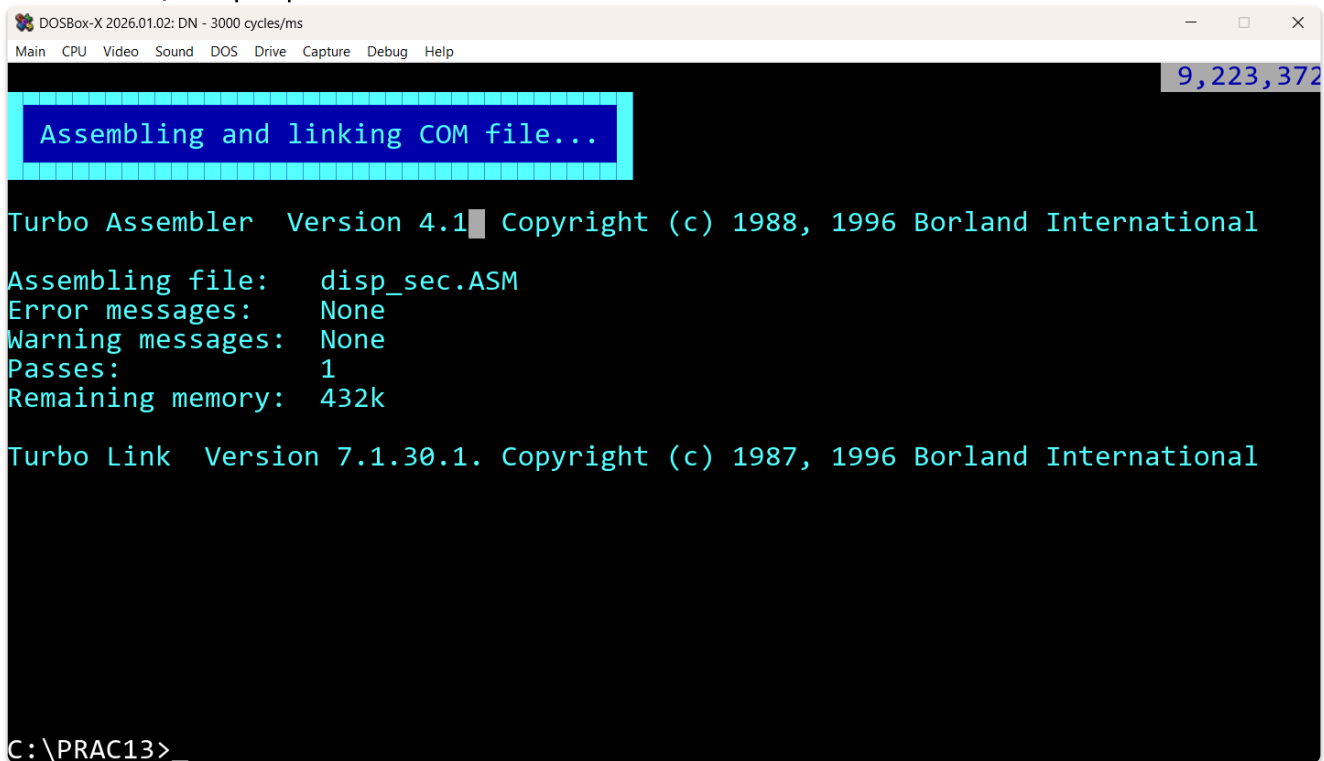
Подключаем новый файл

```
242
243 include video_io.asm
244 include cursor.asm
245 include Kbd_io.asm
246
247 end start
```

Добавление процедуры N_sector для загрузки сектора по введённому номеру

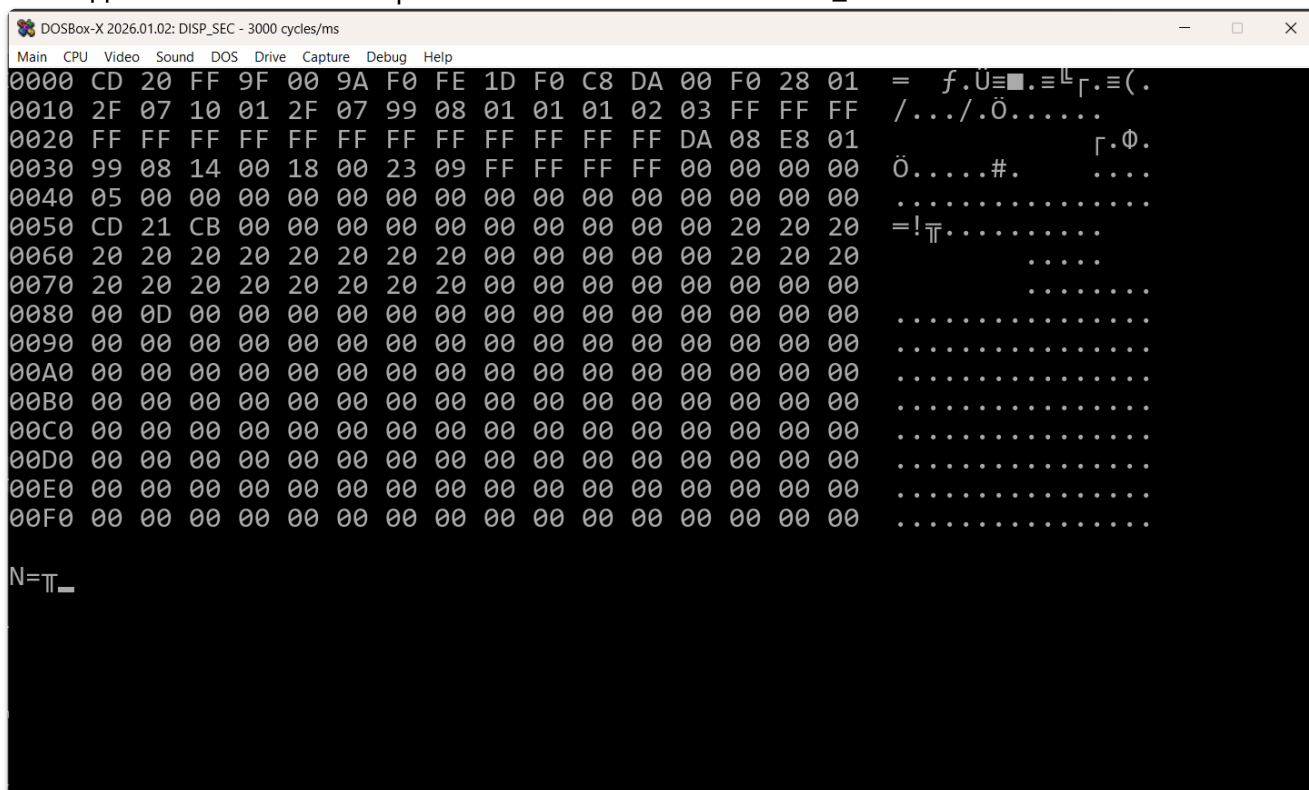
```
171 ; -----
172 ; Загрузка N-го сектора памяти
173 ; N вводится как двухзначное шестнадцатеричное число
174 ; Например: 05 → Address = 0500h
175 ; -----
176 N_sector:
177     push ax
178     push si
179     push di
180
181     call Send_crlf
182
183     mov dl,'N'
184     call write_char
185     mov dl,'='
186     call write_char
187
188     call Read_byte_hex      ; AL = номер сектора
189
190     mov ah,al
191     mov al,0
192     mov [Address],ax        ; Address = N00h
193
194     call Send_crlf
195
196     mov si,[Address]
197     mov di,offset Sector
198     call Copy_sector
199
200     call disp_sector
201
202     pop di
203     pop si
204     pop ax
205     ret
```


Компиляция программы



```
DOSBox-X 2026.01.02: DN - 3000 cycles/ms
Main CPU Video Sound DOS Drive Capture Debug Help
9,223,372
Assembling and linking COM file...
Turbo Assembler Version 4.1 Copyright (c) 1988, 1996 Borland International
Assembling file: disp_sec.ASM
Error messages: None
Warning messages: None
Passes: 1
Remaining memory: 432k
Turbo Link Version 7.1.30.1. Copyright (c) 1987, 1996 Borland International
C:\PRAC13>
```

Вывод начального сектора памяти после вызова Init_sector



```
DOSBox-X 2026.01.02: DISP_SEC - 3000 cycles/ms
Main CPU Video Sound DOS Drive Capture Debug Help
0000 CD 20 FF 9F 00 9A F0 FE 1D F0 C8 DA 00 F0 28 01 = f.U=■.≡Γ.≡(.
0010 2F 07 10 01 2F 07 99 08 01 01 01 02 03 FF FF FF /.../.Ö.....
0020 FF FF FF FF FF FF FF FF FF FF FF DA 08 E8 01 Γ.Φ.
0030 99 08 14 00 18 00 23 09 FF FF FF FF 00 00 00 00 Ö.....#. ....
0040 05 00 00 00 00 00 00 00 00 00 00 00 00 00 00 .....
0050 CD 21 CB 00 00 00 00 00 00 00 00 00 20 20 20 =!T.....
0060 20 20 20 20 20 20 20 20 00 00 00 00 20 20 20 .....
0070 20 20 20 20 20 20 20 20 00 00 00 00 00 00 00 .....
0080 00 0D 00 00 00 00 00 00 00 00 00 00 00 00 00 .....
0090 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 .....
00A0 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 .....
00B0 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 .....
00C0 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 .....
00D0 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 .....
00E0 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 .....
00F0 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 .....
N=T
```

После компиляции была выполнена проверка работы процедур переписки сектора памяти. При запуске программы сначала вызывается процедура Init_sector, которая устанавливает значение переменной Address равным 0000h, копирует начальный сектор памяти в рабочую область Sector и выводит его на экран.

Затем с помощью процедуры N_sector был выполнен ввод номера сектора в шестнадцатеричном формате. При вводе значения 05 программа сформировала

адрес 0500h, скопировала соответствующий сектор памяти в область Sector и вывела его содержимое на экран. После нажатия клавиши была проверена процедура Prev_sector: значение Address уменьшилось на 0100h, поэтому был выведен предыдущий сектор с адреса 0400h.

Также была проверена процедура Next_sector. Для этого через N_sector был введён номер сектора FEh. После вывода сектора FE00h была нажата клавиша, и программа выполнила переход к следующему сектору FF00h. Таким образом, было подтверждено, что процедура Next_sector корректно увеличивает адрес сектора на 0100h.

Отдельно были проверены граничные условия. При текущем секторе 0000h вызов Prev_sector не изменяет значение Address, поэтому программа остаётся на первом секторе. При текущем секторе FF00h процедура Next_sector не должна увеличивать адрес, так как FFh является последним сектором в пределах 64-килобайтного сегмента памяти.

Вывод

В ходе лабораторной работы были реализованы процедуры переписки 256-байтовых секторов памяти в пределах текущего сегмента. Была создана процедура Copy_sector, выполняющая копирование сектора с помощью команды `rep movsb`. Также были добавлены процедуры Init_sector, N_sector, Prev_sector и Next_sector, позволяющие загружать начальный, заданный, предыдущий и следующий сектор памяти.

После компиляции программа была протестирована в среде DOSBox-X. Проверка показала, что программа корректно выводит содержимое выбранного сектора, изменяет значение адреса сектора на 0100h при переходе к соседним секторам и учитывает граничные значения 00h и FFh.